VOSS: Scheduling Attested Session State for Serverless Computing on Volunteer Machines

AALHAD SAWANE, [TODO: institution], [TODO: country]

Short serverless functions spend most of their time waiting for a sandbox to start or for packets to cross a long distance, while machines close to users sit idle. We ask whether those idle consumer machines can serve latency-sensitive requests. Two things make this hard. Owners switch machines off without warning, which destroys every warm sandbox on the host. And owners cannot be trusted: they can read the code and data they run. We observe that remote attestation, in which a machine proves what software it booted, turns a hostile owner into an unreliable one. The owner can still stop or delay the machine, but can no longer read or forge anything. What remains is a scheduling problem over *attested session state*: proof of trust, secret keys and warm sandboxes are slow to create, vanish when a session ends, and widen the attack surface with every host that holds them. We present Voss, a scheduler built around this problem. A slow loop plans where keys and warm state live, with spare capacity for lost hosts. A fast loop places each request and duplicates risky ones. Failures are detected from missing progress rather than missing heartbeats, and schedulers are replaced without consensus among volunteers. Workers reach their scheduler through a provider-hosted publish/subscribe broker, because volunteer addresses are neither stable nor reachable. In a deterministic simulator calibrated on a TPM-equipped machine, Voss meets a 3 s tail-latency target in 9 of 9 churn-and-adversary settings and 13 of 16 substrate-and-load settings, where the strongest simple baseline meets it in none. Several designed components do not pay off, and we report those too.

CCS Concepts: • **Computer systems organization** → **Cloud computing**; • **Security and privacy** → *Trusted computing*; • **Networks** → Network protocols.

Additional Key Words and Phrases: serverless computing, volunteer computing, remote attestation, scheduling, publish/subscribe, tail latency

ACM Reference Format:
Aalhad Sawane. 2026. VOSS: Scheduling Attested Session State for Serverless Computing on Volunteer Machines. *ACM Trans. Internet Technol.* 1, 1 (October 2026), 30 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 INTRODUCTION

Serverless platforms run small pieces of customer code, called *functions*, whenever a request arrives. Most of these functions finish in well under a second [43], so the useful work is tiny compared with everything around it. Before a function can run, the platform must create an isolated environment for it and load its language runtime. This is the *cold start*. The request must also travel to wherever the function happens to run. Providers hide the cold start by keeping environments alive in their datacenters, which holds memory around the clock [24, 43]. They can do little about distance.

Meanwhile, millions of home and office computers are close to users and idle for most of the day. Volunteer computing has used such machines for two decades [6, 7], but only for batch jobs with deadlines of hours. We ask a harder question: can volunteer machines answer requests that must finish in a few seconds?

Two things stand in the way. The first is **churn**. An owner can switch off, reboot or reclaim the machine at any moment, and everything the platform prepared on it disappears: warm environments, cached code and the secret keys it was given. For a function that runs for tens of milliseconds, preparing this state again costs far more than the function. The second is **distrust**. The owner controls the hardware and the operating system, so the owner can read the customer's code and

Author's address: Aalhad Sawane, [TODO: institution], [TODO: country], aalhadsawane@gmail.com.

2026. ACM 1533-5399/2026/10-ART
https://doi.org/10.1145/nnnnnnn.nnnnnnn

data, return forged results, or walk away at the worst moment. The two feed on each other. The usual cure for distrust is to check a machine before using it, and that check takes time. Churn throws the result away each time a session ends.

Earlier work solves only parts of this. Volunteer-computing systems tolerate untrusted machines by running every task several times and comparing answers [6]. That multiplies cost, works only for tasks with a single right answer, and still shows the input to whoever runs it. Serverless schedulers avoid cold starts well [24, 43] but assume trusted machines that stay up. Confidential serverless designs protect customers from the platform operator [1, 9] but run on provider servers and treat the trust check as a one-time setup. And the standard way to notice a dead machine, listening for its periodic "I am alive" message (a heartbeat), fails against an owner who keeps sending heartbeats while throwing work away. Such a quiet failure is a *gray failure* [28].

Our starting point is that proving what software a machine runs changes the fault model. If the customer's code and the scheduler's own code run only inside a locked, measured software image, and a verifier has checked hardware-signed evidence of that image (*remote attestation*), the owner cannot read, forge or tamper with anything inside. The owner keeps only the power switch and the network cable. A hostile owner can therefore do exactly what churn already does: stop, or delay. This failure class has a standard name, *omission and timing failures* [10, 19], and a volunteer platform has to tolerate it anyway. One set of mechanisms then covers crashes, churn and malice.

That reduction exposes the real scheduling problem. A machine is *attested* for one session. It becomes *provisioned* for a customer once that customer's key service has released a secret key to it, and *warm* for a function once a ready environment is waiting. We call these together *attested session state*. It is *slow to create*: on our measurements a powered-off machine needs 65 s to become an attested worker, thousands of times longer than resuming a warm environment. It is *perishable*, since every session end destroys it. And it is *exposing*, since each machine holding a customer's key is one more place the key could leak. Choosing where this state lives is a scheduling decision with a price, and it cannot be left to the per-request path.

Voss is a scheduler built around this view (Figure 2). It has three ideas. First, it separates *where state lives* from *where a request runs*. A slow loop decides which machines hold which keys and warm environments, with spare capacity so that losing a few machines does not hurt. A fast loop then picks a machine for each request and duplicates the risky ones. Second, it detects failures from missing *progress*, meaning work accepted but never finished, instead of missing heartbeats, and it replaces a failed scheduler without asking volunteers to agree on anything. Third, it records work with signed receipts rather than a blockchain. Because volunteer machines sit behind home routers and change address often, machines reach the scheduler through a provider-hosted publish/subscribe broker with one topic per cluster (Section 6).

Our contributions are: (1) showing that attestation reduces hostile owners to omission and timing failures, and stating the resulting problem of managing attested session state; (2) a two-timescale scheduler whose planner keeps coverage even if any f machines fail together; (3) a communication architecture for unstable, unreachable volunteer addresses, and a comparison of volunteer-hosted and provider-hosted schedulers; (4) fault tolerance without agreement among volunteers; (5) a deterministic, calibrated evaluation with hypotheses and decision rules fixed in advance, including what did not work.

Planning is the largest lever: it lowers tail latency by 54–59% against no planning, and progress-based detection keeps 99.95% of requests succeeding when 20% of owners drop work, where heartbeat-only detection falls to 96.8%. Voss meets a 3 s tail-latency target in most settings we map, and the strongest simple baseline never does. Yet some designed components do not earn their complexity: geographic clustering was slower than one scheduler, an exact solver was not

distinguishable from a greedy heuristic, and a commercial-style reference is still faster. All results come from simulation and one calibration machine (Section 10.1).

2 BACKGROUND AND PROBLEM

Terms. A *tenant* is a customer of the platform: the developer or company that writes functions. An *invocation* is one call of a function. The *provider* runs the platform, and a *volunteer* or *owner* owns a machine that donates capacity. A *sandbox* is a small virtual machine (microVM) that runs one function in isolation, and it is *warm* when it is started and idle. A tenant's *secret* is a cryptographic key that only the tenant controls. It decrypts the tenant's code and the saved memory images (snapshots) of its sandboxes, so whoever holds it can read that tenant's code and data.

2.1 Where the time goes

The latency of an invocation is a sum of waiting times: choosing a region and a machine, queueing for a slot, making the machine able to run this tenant's code, starting a sandbox, moving data, and running. For functions of tens to hundreds of milliseconds, starting the sandbox dominates unless one is already warm. Platforms therefore keep sandboxes alive or restore them from snapshots [8, 14, 24, 43]. We distinguish four start-up levels for a function on a machine: **L0**, nothing cached; **L1**, code and file system on local disk (boot and initialise); **L2**, a snapshot on disk (restore it); **L3**, an idle warm sandbox in memory (just send the request). Higher levels cost memory or disk, so choosing which functions to keep at which level where is already a scheduling problem.

Volunteer machines add a term datacenters do not have. Before a machine may run tenant code it must prove what software it runs, and before it may decrypt a tenant's code it must be given that tenant's key. Figure 1 puts these costs on one scale, using a consumer laptop with a TPM, a hardware security chip (Section 8.2). Resuming a warm sandbox takes milliseconds. Booting a fresh one takes about 154 ms if its files are in memory and 1.45 s if they come from a hard disk. Getting from power-off to an attested worker takes 65 s (median of 9 boots): 21 s for firmware, boot loader and kernel start, and 44 s more until the verifier accepts the machine's proof. These are upper bounds, because we ran the appliance on a hard disk and in a simplified mode. Even so, trust costs thousands of times more than resuming a sandbox, so *where* it is established cannot be left to the per-request path.

2.2 Proving what a machine runs

We use the vocabulary of the IETF attestation architecture [12]. While the machine (the *attester*) boots, each stage (firmware, loader, kernel) is hashed into registers of its TPM, so the final values are a fingerprint of everything that ran [44]. When asked, the TPM signs those values together with a fresh random number from the checker. This signed object is the *quote*. The *verifier* checks the signature, follows the certificate chain back to the chip manufacturer, and compares the fingerprint with known-good values. A *relying party*, here the tenant's key service, hands over a secret only to a machine with a good result [40]. Two facts matter for scheduling: a quote shows what ran at one moment, so evidence goes stale and must be refreshed, and attestation says nothing about availability, since a machine that passes can still stop working.

2.3 Volunteer machines and how they fail

A volunteer machine alternates between off and a *session*, an interval in which it is booted into the platform. Session lengths are heavy-tailed and follow the time of day [31]. A session can end gracefully, with notice, or abruptly, sometimes for a whole city at once. We classify failures in the standard way [10, 19]: a crash stops the machine, an *omission* is a result that never arrives, and a *timing* failure is a result that arrives too late. Churn produces crashes. An owner who cannot forge

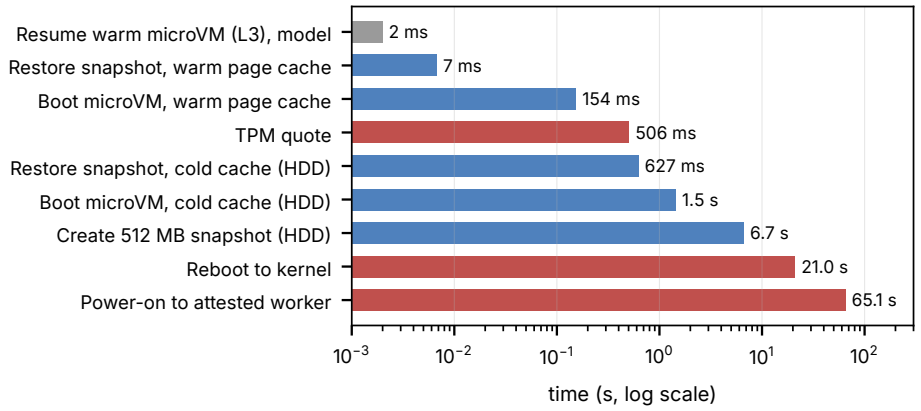


Fig. 1. Time to reach each state on the calibration machine, log scale. Blue: sandbox start-up. Red: becoming an attested worker. Grey: model value, not measured. *Warm cache*: the file is already in memory; *cold*: it is read from the hard disk. *Reboot to kernel* is the physical machine's firmware, loader and kernel start.

anything can still choose *when* to omit or delay, selectively, while the machine keeps reporting that it is healthy. That is the gray failure of Section 1, and it defeats heartbeat detectors, including adaptive ones [26].

2.4 The problem

The input is a stream of invocations, each with a tenant, a function, a service class (synchronous, asynchronous, or low-priority filler), a deadline and an *idempotence* flag. A function is idempotent if calling it twice has the same effect as calling it once. The platform retries and duplicates only idempotent functions; for the others a lost result is a failure. Asking for idempotence is not new. Every major platform already hands it to developers, though the underlying guarantee differs by call path. AWS Lambda retries a failed asynchronous invocation twice by default and warns that a function may receive the same event more than once [3]. For a direct synchronous call it does not retry at all, and leaves the retry, with the same warning about repeats, to the caller [4]. Google's event-driven functions run at least once per event, retried functions must be idempotent, and plain HTTP functions are not retried by the platform [25]. Azure documents at-least-once delivery for Event Hubs triggers and asks for idempotent consumers [34], and Cloudflare Queues deliver at least once [15]. So the guarantee is at least once on event and asynchronous paths, and *not* on direct request–response paths, where the client retries instead. On both, a duplicate arrives sooner or later, from the platform or from the client, so developers are told to write idempotent code. Voss relies on the same contract. It goes one step further than commodity platforms by also retrying and hedging *synchronous* calls for the functions that declare it, because failures on volunteer machines are common. The machines are a changing population with slots, memory, disk, a location and a session that can end at any time. The decisions are which machines to attest and provision for which tenants, which functions to keep at which level where, and which machine runs each invocation. The goals are: **G1**, a hostile owner learns no tenant secret and forges no result; **G2**, tail latency stays within target despite churn; **G3**, a machine that drops work or a scheduler that stalls is bypassed within seconds without trusting liveness signals; **G4**, a machine cannot claim work it did not do. We do not defend against physical attacks on the TPM, flaws in the appliance image, side channels, or failure of the provider's own control plane (Section 4.1).

3 SYSTEM DESIGN

This section follows the problem through the system: who takes part, what each party is trusted for, and what happens to one invocation.

3.1 Three principles

Attestation turns distrust into unreliability. Everything that touches tenant secrets, or makes scheduling decisions, runs inside the attested image. The rest of the system then only has to tolerate machines that stop or stall (Section 4). **Trust is a resource to be planned.** Attestation, keys and warm state have a cost, a lifetime and an exposure price, so they are planned more slowly than invocations are placed (Section 5). **Liveness is a hint; progress is evidence.** Signals from untrusted machines, such as heartbeats, drain notices (an owner announcing they will leave) and self-reported health, are never enough. Failures are detected where work is observed by a party that can be trusted (Section 7).

3.2 The parties

Figure 2 shows the components, with colour marking who is trusted for what.

The provider's control plane is assumed reliable and is trusted for availability and accounting. It never holds tenant keys. *Routers* receive invocations, pick a cluster, keep a pending copy until a worker accepts, watch for progress, and write the receipts that are the ground truth of work. The *orchestrator* groups workers into geographic *clusters* of tens of machines, which keeps each planning problem small and distances short, and appoints one scheduler per cluster. A *verifier* checks attestation evidence and issues session certificates, a *registry* serves encrypted code bundles, and a *broker* carries control messages (Section 6).

Tenants run, or delegate to a service they trust, a *key service* that releases their secrets only to machines that attest well. The provider does not hold tenant keys, so this service is not provider-operated.

Volunteer machines reboot into the *attested appliance*, a locked, measured operating-system image containing the worker agent, the Firecracker microVM monitor [2] and the scheduler code. Each *worker* runs every invocation in its own microVM and keeps an encrypted cache of code and snapshots. The *scheduler* is a role, not a machine: the same image runs everywhere, so any worker can be promoted without booting anything, and one worker per cluster is appointed to schedule. We study this placement because it needs no per-cluster provider component and is the harder case for fault tolerance. Section 6.4 discusses the alternative. While the appliance runs, the owner cannot use the machine, so donation means a scheduled or dedicated session. The owner still controls power, network and firmware settings, which is why the scheduler's *host* can omit or delay even though its *code* cannot misbehave.

3.3 Life of an invocation

A router chooses a cluster by round-trip time, health and the capacity already provisioned for the tenant, and sends only *metadata* to its scheduler. The scheduler picks a worker, possibly with a duplicate (a *hedge*) on a second one. The router then sends the payload to the worker over a connection that ends inside the attested appliance, so the scheduler never sees payloads, results or keys. If the worker is not yet provisioned for the tenant, it asks the key service for the secret, climbs the start-up levels and runs the function. The result goes back to the router with a receipt signed by the worker's session key (Section 7.3), and the client's latency ends there. Every term of the latency is logged separately, which lets us attribute latency to causes in Section 9.

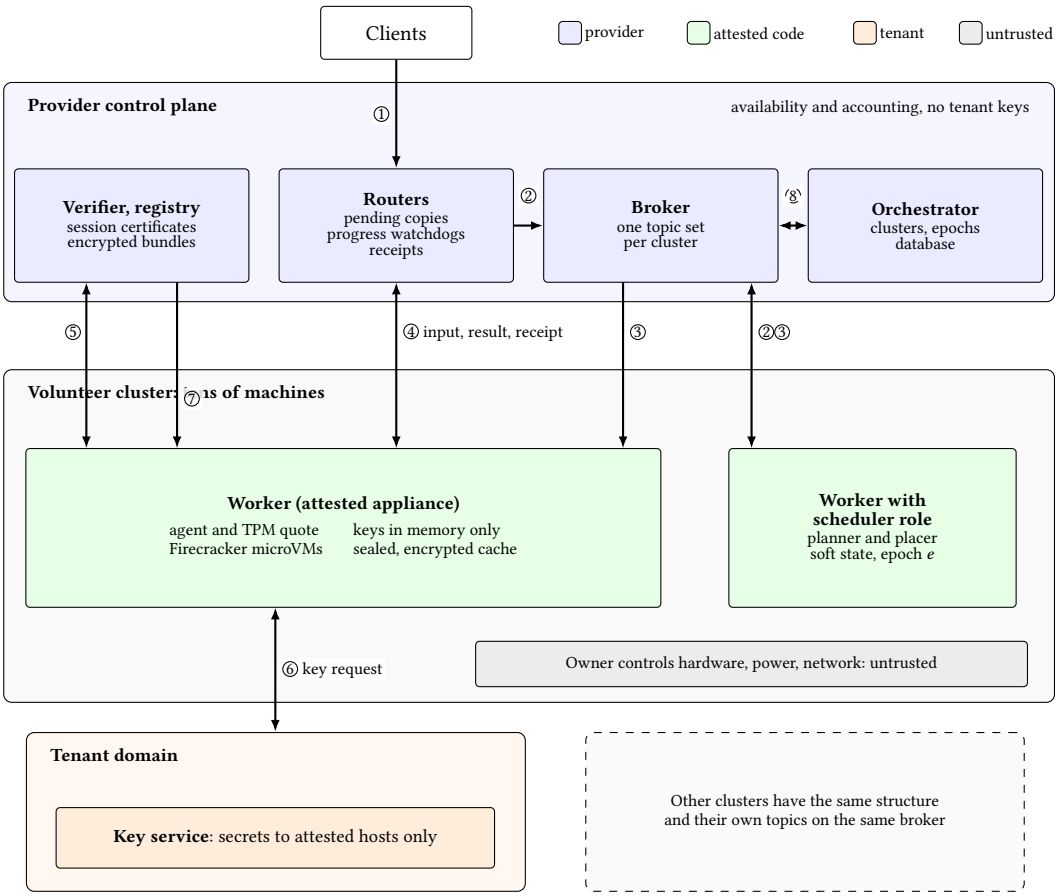


Fig. 2. The Voss architecture. Numbers follow one invocation: (1) a client calls a router; (2) the router sends metadata to the cluster's scheduler through the broker; (3) the scheduler dispatches to a worker; (4) the router sends the input and the worker returns the result and a receipt; (5) the worker proves itself to the verifier; (6) it asks the tenant's key service for the secret; (7) it fetches code from the registry; (8) the orchestrator appoints schedulers (dashed, off the request path).

3.4 Sessions and start-up levels

A worker alternates between OFF and a session. It becomes schedulable only after booting and being attested, which takes 65 s from power-on on our machine. A session ends abruptly or gracefully, and either way every secret and warm sandbox is discarded. Disk caches survive as ciphertext and work again after the next attestation. Within a session, a function sits at one of the start-up levels of Section 2, and a separate flag records whether the machine holds the tenant's key. These two facts, a trust step per session and a key flag per tenant, are what the simulator's generic lifecycle engine reads from data for every trust substrate (Section 4.5).

3.5 Rules for state and messages

Four rules keep the request path fast and failures recoverable. Every interaction is a typed message. The operational database is private to the control plane and never on the request path. Live state

Table 1. Adversaries, what neutralises them, and what is left.

Adversary	Capability	Neutralised by	What remains
Volunteer owner	controls hardware, power, network; may boot modified software, fake a TPM, replay, delay, drop	measured boot and attestation; manufacturer-certified keys; locked image with no login; disk sealed to the measurement	**omission and timing**
Network attacker	watches and changes traffic	encryption ending inside attested parties	traffic metadata
Other tenants	malicious function code	microVM isolation	side channels

stays where it is born (queues, caches and keys on workers, pending work in routers, estimates in the scheduler) and is rebuilt from the workers after a scheduler change (Section 7.2). And every channel ends inside an attested party, with the session certificate as the credential.

4 TRUST AND ITS COST

The design rests on one claim: attestation reduces a hostile owner to an unreliable one. This section states the claim, shows how a machine earns trust and a tenant's key, and defines the exposure that keys create.

4.1 Threat model

We trust the CPU, its TPM and its memory protection; the measured appliance image; each tenant's own key service; and the provider's control plane, for availability and accounting only (Table 1). Out of scope are laboratory-grade physical attacks such as voltage glitching of firmware TPMs [30], flaws in the appliance, side channels and traffic analysis, and failure of the control plane. An owner who can read memory while the appliance runs (cold-boot or bus probing) is outside the claim below unless the host encrypts memory. We express that residual risk only as a policy number, the *assurance* of Section 4.4.

Property 1 (threat reduction). Under these assumptions, every behaviour of a worker or scheduler host that other components can observe, and that software can reach, is a crash, an omission or a delay. The owner cannot run code other than the measured image, because it would fail attestation and receive no certificate and no key. The owner cannot read or change the image's memory or sealed state, and results and receipts are signed with a key that never leaves the image. What is left is power, the network path and the timing of when to stop answering. So the mechanisms that churn already forces on us also cover malice, and redundancy is no longer needed for integrity.

Other mechanisms fall short. Memory encryption alone attests nothing and leaves plaintext visible to the operating system. Running every task several times and voting triples cost. Verifiable computation is far too slow for functions of milliseconds. Confidential VMs isolate best but do not exist on consumer hardware, so we treat them as one substrate among several (Section 4.5). A blockchain would protect against no adversary here, because a receipt signed by an attested key can already be checked by anyone (Section 7.3).

4.2 Becoming an attested worker

The appliance is a signed kernel image with a read-only, integrity-checked root file system, the worker agent and the scheduler code, with no owner login. The partition that holds caches is

encrypted with a key sealed to the expected measurement. A session starts when the owner reboots into the image and ends on reboot, power loss, a drain request or failure.

Figure 3 shows the exchange, which follows Keylime-style attestation [40]. The agent creates a *session key pair* inside the image and binds its public half into the quote. The verifier checks the quote and certificate chain, compares the boot record with reference values and, if all is well, issues a *session certificate* with the machine's identity, the session public key, an assurance score and an expiry. Every channel in the system uses this certificate, so no untrusted operating system sits on any path. Two properties matter for scheduling. *Trust is perishable:* the certificate expires when the session ends or one validity period after the last good quote, is extended by periodic re-quotes, and is revoked if one fails. And *identity costs a machine:* a worker's identity is the hash of its TPM's endorsement key, so many identities cost real hardware [22].

4.3 Getting a tenant's key

Attestation shows that a machine runs the appliance, not that it may see a particular tenant's data. The tenant's key service holds a policy, for instance "accept appliance versions at or above this one, optionally with a fresh quote". Making worker j *provisioned* for tenant t takes one round trip: the worker sends its certificate, the key service checks the policy and returns the key wrapped to the session public key, and the worker keeps it in memory only. This trip, plus decrypting bundles, is the provisioning term in the latency. Code bundles are encrypted with the tenant key, so any worker may store and forward them but only provisioned workers can use them. Snapshots contain decrypted memory, so they are written only by provisioned workers, shared only with other provisioned workers of the same snapshot class, and counted in exposure.

Whom the tenant must trust for confidentiality depends on the policy. If the key service accepts any certificate the verifier issued for an approved measurement, the verifier is trusted for confidentiality. If the policy demands a fresh quote, the key service checks it itself and does not depend on the verifier, at the price of one more quote. We assume the verifier acts as certificate authority, with its root distributed in the image and to key services. We do not evaluate this infrastructure, and the simulator charges one fixed calibrated cost for the policy check.

4.4 Exposure and assurance

Every provisioned machine is a place from which a tenant's secret could leak if our trust assumptions fail. We define the *key exposure* of tenant t as

$$E_t = \sum_{j \in \mathrm{KH}(t)} (1 - a_j), \tag{1}$$

where $\mathrm{KH}(t)$ is the set of machines holding t's key or decrypted artifacts and $a_j \in [0, 1]$ is a machine's *assurance*: a base value for its trust substrate times a factor for firmware policy, fixed when the certificate is issued. Exposure is a transparent proxy for attack surface, not a probability of compromise, and base assurances are policy parameters, not measured truths. We therefore report latency, exposure and robustness together as a trade-off front, never as one weighted number. Tenants set a sensitivity that weights exposure and an assurance floor below which a machine never gets their key.

4.5 Substrates

A *substrate profile* is data that tells one generic lifecycle engine how long trust takes, how long it lasts, what each start-up level costs and what assurance the host carries, so the scheduler is independent of how trust is established. We model five. The main one, the TPM-attested appliance on consumer hardware, uses costs we measured (Section 8.2). Two confidential-VM variants for

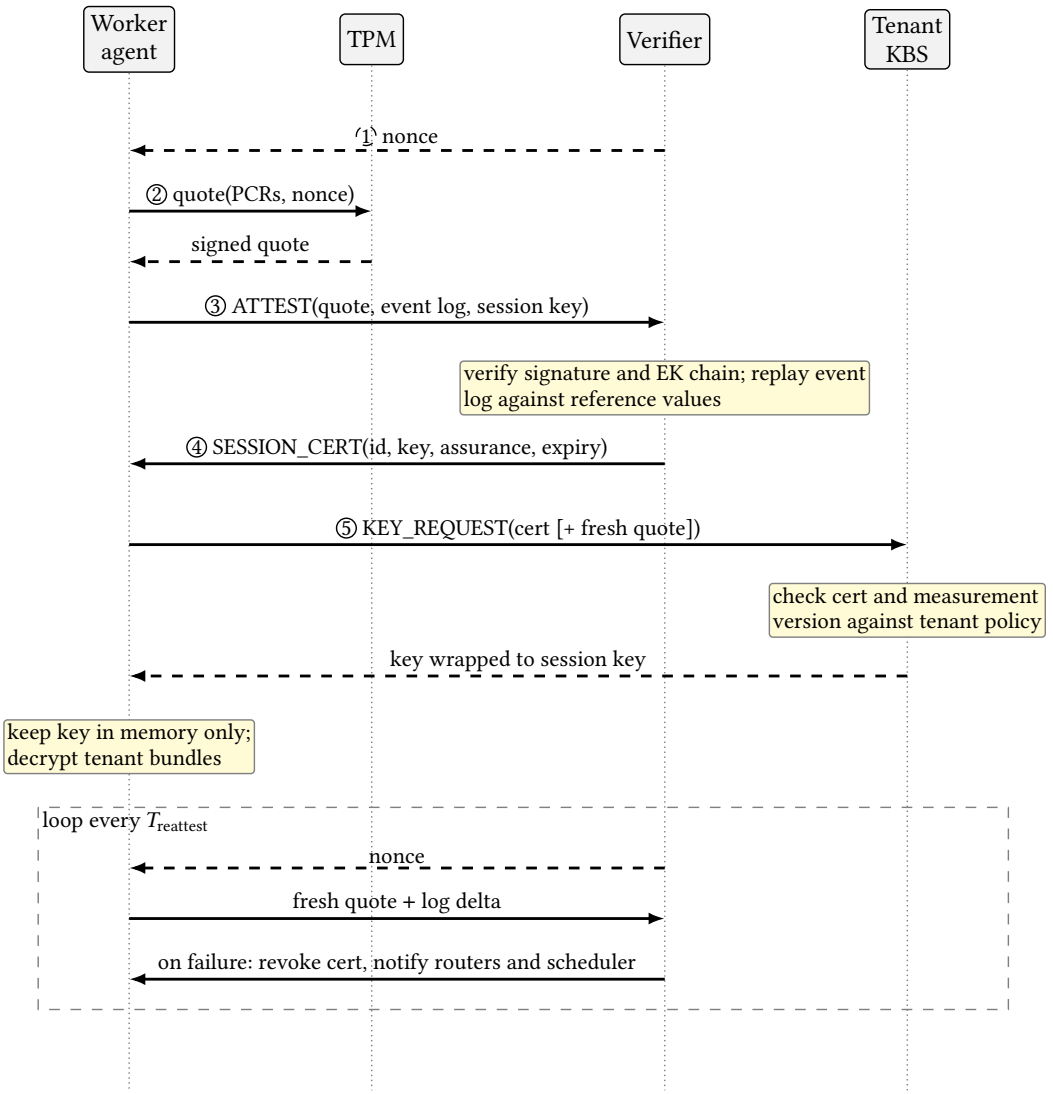


Fig. 3. Attestation and key provisioning for one worker. The session key is bound into the quote, so the certificate belongs to this appliance instance only.

server CPUs use ranges from the literature [5, 27, 33, 35]. One models a measured hypervisor under the owner's operating system. The last, none, is the unattested reference that baselines use.

5 SCHEDULING ATTESTED SESSION STATE

Section 4 made trust a resource with a price. This section asks who should hold it, and when. Each cluster (tens of machines) has one scheduler with two jobs: decide which machines hold which attested session state, and assign every invocation to a machine.

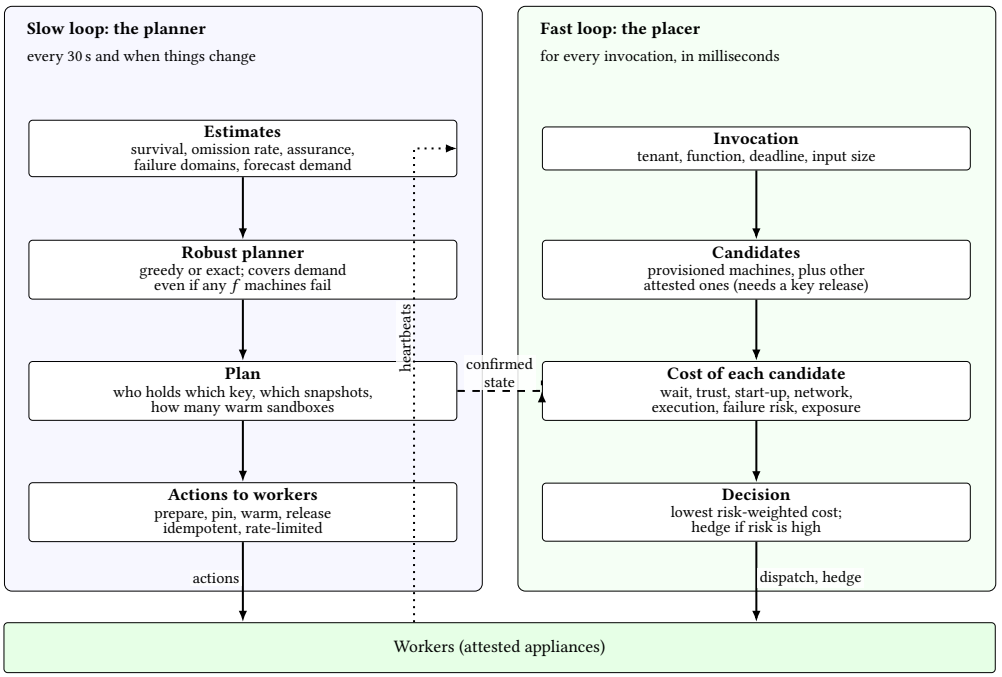


Fig. 4. The two loops inside a scheduler. The planner decides where keys and warm state live and sends actions to workers. The placer picks a machine for each invocation. Workers confirm plan actions in their heartbeats, so the placer never relies on a machine that is not yet ready.

5.1 Why two timescales

Could one per-invocation scheduler do both jobs? Three reasons say no. First, the security cost belongs to a group of machines. A key released to a machine stays for the whole session, so what matters is how many *different* machines hold a tenant's key, and that count does not add up one placement at a time. Second, robustness is a property of the group: whether demand is still covered if any f machines disappear cannot be read off a single placement. Third, establishing state takes seconds to tens of seconds (Figure 1), far beyond a per-request budget, so it must happen before demand arrives. Placement is the opposite: urgent, and a small choice among a few candidates.

So Voss uses two loops (Figure 4). The **slow loop**, the *planner*, runs every 30 s and whenever things change materially. It decides which machines hold which tenant's key, which snapshots sit on whose disk, how many warm sandboxes each machine keeps, and how many slots are reserved for each tenant. The **fast loop**, the *placer*, runs for every invocation and chooses from machines the planner has prepared. The 30 s period is a design choice, not a measured optimum: plan actions take from a few hundred milliseconds to several seconds, so the period must exceed that, and sessions last minutes to hours, so it must be far shorter. We did not sweep it.

5.2 What each loop minimises

What we care about in the end is the *SLO violation*: an invocation fails or finishes after its deadline, which for synchronous functions is five times the warm execution time plus 50 ms. The two loops treat violations differently.

The placer minimises each invocation's own risk. For invocation i on candidate machine j it computes

$$J_{ij} = w_i \, P_{ij}^{\text{miss}} + \varepsilon \, \mathbb{E}[T_{ij}] + \lambda_{\text{exp}} \, x_{ij}. \tag{2}$$

P_{ij}^{miss} is the probability that i violates its SLO on j: a machine that fails first counts as a violation, and otherwise the predicted finish time is compared with the remaining deadline. $\mathbb{E}[T_{ij}]$ is the expected latency, and x_{ij} is the exposure added if j does not yet hold the tenant's key. The default weights give J a plain meaning: one second of expected latency counts like a 5-point rise in violation probability, and giving a medium-sensitivity tenant's key to a new machine of assurance 0.8 costs the same as a 20% chance of a violation. We chose per-invocation risk over a P99 target because P99 is a statistic of the whole population and cannot be assigned to one decision, whereas each invocation's own miss probability is the quantity whose average is the miss rate.

The planner minimises a surrogate. At planning time the invocations do not exist, only a forecast of demand. Minimising violations would also be circular, because they depend on the placer's later choices, which depend on the plan. Violations are rare (1–5%), so an objective built from them is noisy. And they are not the only cost: warming everything everywhere would minimise violations and hand every tenant's key to every machine. The planner therefore minimises cost *subject to covering demand*. Demand is the number of slots that, assuming Poisson arrivals, covers the forecast load 95% of the time, so nearly every invocation can start at once, and coverage must hold even after the f largest allocations are lost. Uncovered demand is charged far more than memory or exposure, so coverage is bought unless the budgets make it impossible, in which case the shortfall is reported. This is a design rationale, not a theorem. Simulated arrivals have bursts that the Poisson model misses, so realised coverage dips during bursts, and the experiments measure the violation rate itself.

5.3 What the scheduler knows

For every number a decision uses, a reviewer should be able to tell whether it was observed, learned, forecast or assumed. *Observed* inputs (busy slots, cached state, provisioned tenants, session expiry, assurance) are reported by workers every 2 s, plus the scheduler's own dispatches since. *Learned* inputs (session survival, omission rate, execution time per function and machine) come from online estimators fed only by past outcomes, and *forecasts* (demand over the next 60 s) are computed from them. *Calibrated or configured* inputs (start-up and trust costs, network times, host speeds, weights, tenant policy) are fixed before the run and not re-learned. The scheduler never sees the true end of a session, an invocation's true run time, a host's true behaviour or future arrivals. The code base enforces this by forbidding the policy layer from importing the simulated world. The speculative part is the last group: the scheduler assumes start-up costs and network times equal their calibrated values and that declared host speeds are exact. A deployment would measure these online. This prototype does not (Section 10.1).

5.4 The slow loop: planning keys and warm state

Every 30 s the planner looks at the live, non-draining machines in its view (at most 40), and at the 20 busiest tenants and 50 busiest functions. The long tail stays on the unprovisioned path, which keeps the problem small; the evaluation workload has six tenants and 12 functions, so these caps never bind. Keys belong to tenants, so the planner first decides which machines hold each hot tenant's key, then which of its functions stay warm there. The result is a *plan* valid for the next 60 s.

Robust coverage. The planner requires that provisioned, available capacity still covers forecast demand after removing the f largest allocations:

$$\sum_j e_j \, c_{tj} \;-\; (\text{the } f_t \text{ largest } c_{tj}) \;+\; u_t \;\geq\; q_t. \tag{3}$$

Here c_{tj} is the number of slots of machine j reserved for tenant t, e_j the estimated chance that j survives the horizon and does not omit, q_t the demand to cover, and u_t the shortfall that is charged. This is a standard budgeted-robustness constraint [11], written so the integer program stays linear. It needs no model of the adversary, because it assumes the f_t worst machines fail together, even ones with a perfect record. By default f_t is a tenth of the cluster, rounded up, and tenants may opt down. Allocation also respects each machine's slots, memory and disk, a cap on how much of a tenant's capacity may sit in one failure domain (one network provider or city), the assurance floor, and attestation freshness.

Two ways to solve it. The exact formulation is a mixed-integer program with about a thousand variables per cluster, which we solve with HiGHS [29] under a time limit. The *greedy planner* serves hot tenants in decreasing order of demand times cold penalty. For each it repeatedly adds the machine with the best survival-times-assurance per unit of memory and exposure cost, until coverage with the f_t largest removed reaches the demand. It then adds warm sandboxes and snapshots in decreasing order of start-up time saved per megabyte. Greedy planning takes about 8 ms per solve. In a pilot, the exact program needed 0.3–1.1 s for 10 hosts, 4.7–5.0 s for 40 and 17–24 s for 80. Before the experiments we fixed a decision rule: if the exact program improves P99 by less than 5% over greedy in every regime, greedy is the method and the exact program a reference (Section 9.3).

From plan to action. A plan is re-solved only if the situation changed materially. Otherwise it is compared with its predecessor and turned into idempotent, rate-limited actions sent to workers: prepare (fetch the key and decrypt bundles), pin a snapshot, warm a sandbox, release. Workers confirm completion in heartbeats, and the plan counts only for state they have *confirmed*.

5.5 The fast loop: placing each invocation

For each invocation the placer sees only metadata (function, tenant, remaining deadline, input size, idempotence), the scheduler's current view and the plan. It considers the best $K = 12$ machines by a cheap lower bound, from the confirmed provisioned machines and from other attested machines (the *unprovisioned path*, which needs a key release and usually a start-up). Draining, soon-to-expire, low-assurance or memory-short machines are out. For each candidate it estimates and logs five delays: the wait for a slot, the trust cost (zero if the key is already held), the start-up cost, the network time and the execution time. Completion time is modelled as a shifted log-normal over these parts, and failure probability is the chance that the machine ends its session or omits before the run is over. A synchronous invocation takes the lowest J immediately. Asynchronous and filler work arrives in 0.2 s windows and is assigned by minimum-cost matching. All baseline placers use the same candidates and cost model, so differences come from the decision rule alone.

Admission, hedging and retries. If every candidate is almost sure to miss and even the shortest wait exceeds the remaining deadline, the system is overloaded: a synchronous request is rejected and an asynchronous one deferred. Misses caused only by a cold start are admitted, because they warm the caches later requests need. If the best candidate's miss probability is above 0.2, the function is idempotent and a budget of 10% of placements allows it, the placer sends a duplicate to the best machine in a *different failure domain* and cancels the loser when the first result arrives [21]. After a watchdog or failure the invocation is placed again with the failed machine excluded, within

a budget. Receipts make duplicates safe for integrity but not for side effects, so hedges and retries apply only to idempotent functions.

5.6 Estimators

Four small estimators feed both loops, trained online from outcomes the scheduler can see. *Survival* is a Weibull distribution of session length per machine tier, refitted every 60 s with a time-of-day adjustment. Drain notices can only shorten predicted life, since they may be fake. *Omission* is a Beta posterior over accepted invocations that return no result, and a change-point test (CUSUM) resets a machine whose behaviour shifts, which defeats owners who behave until they defect. *Execution time* is a Bayesian regression per function with per-machine offsets [42], and *arrivals* are forecast per tenant and function from counts in 10 s bins. Each keeps a few numbers, not a table of past runs, and lives in the scheduler's memory with a checkpoint at the control plane every 30 s. The planner's robustness does not depend on the omission estimator being right; the estimator only reduces how often robustness has to be used.

5.7 Clusters and routing

The orchestrator groups workers geographically by clustering on round-trip times, targeting 30 machines per cluster (10 to 60), requires enough failure domains for a robust plan, and rebalances with hysteresis, since a moved worker loses its provisioned state. A router scores each cluster for an invocation by the delay it expects there, in seconds: the round trip, the cluster's scheduling overhead and backlog, a penalty for a high recent miss rate, the key-release time if the cluster has no capacity provisioned for the tenant, and the cold-start time if it has no warm sandbox. Spilling over to another cluster is thus a decision with an explicit price.

6 REACHING THE SCHEDULER

Section 5 assumed that routers can send work to a cluster's scheduler, and that the scheduler can send plans to its workers. But the scheduler is a role on a volunteer machine. Can anyone actually reach it?

6.1 The problem: volunteer addresses

Volunteer machines sit behind home or office routers. Such a router lets a machine open connections to the outside (the machine *dials out*), but it drops connections that arrive from outside, because it does not know which machine inside they are for. On top of that, the addresses of home connections change when a router restarts or the internet provider reassigns them. A volunteer-hosted scheduler has both problems. Three things must work. Workers must reach their scheduler. Routers must reach it too. And when the scheduler changes, because of a crash or a planned handover, everyone must find the new one.

6.2 Why request and response do not fit

The usual way for two programs to talk, HTTP or gRPC, assumes that a server listens at an address that clients know. Here the server is a home machine with no inbound path and an address that lives for one session. Every scheduler change would force every worker and router to learn a new address and reconnect at the same moment. Punching holes through the routers on both sides helps only some of the time, and the pairs that fail need a relay anyway. So we do not make the scheduler a server. We use the relay idea directly and make it the architecture.

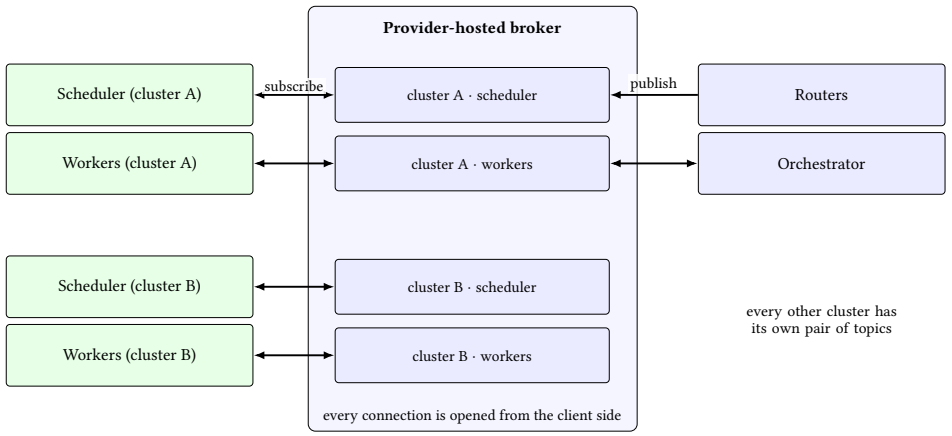


Fig. 5. Control traffic between machines goes through topics on a provider-hosted broker, one set per cluster. Every connection is opened from the volunteer side. Invocation payloads do not use the broker: workers also keep an outbound stream to their regional router.

6.3 Our design: a provider-hosted publish/subscribe broker

The provider runs a *broker*, a public service with a stable address. Every party dials out to it and keeps the connection open. Messages are not sent to machines. They are published to named *topics*, and the broker forwards each message to everyone subscribed to that topic (Figure 5). Voss gives each cluster its own topics, named by cluster id. The scheduler subscribes to its cluster's scheduler topic. Each worker subscribes to its own topic and to a cluster-wide one. Routers publish to the scheduler topic. Nobody needs to know anybody else's address. The only thing they must share is the cluster id, which the orchestrator tells them when it assigns them.

We take MQTT [37] as the candidate protocol. It was built for devices on unreliable networks, clients always dial out, and it offers delivery guarantees that match our messages: at-least-once delivery is enough for plan actions, which are idempotent, and for dispatches, which carry the scheduler's epoch (Section 7.2). We do not depend on MQTT in particular. Any broker with topics and outbound connections gives the same architecture, and the choice of protocol is an implementation detail.

The design has four consequences.

- **Failover is simple.** A new scheduler subscribes to the same topic. The topic name does not change, so there is no address to hand out and no reconnect storm. Workers learn only the new epoch and certificate from the appointment message.
- **The broker is trusted for availability only.** Messages between attested parties are sealed with the keys in their session certificates, so the broker sees topic names and ciphertext. It cannot forge a dispatch, because workers accept dispatches only from the current epoch and certificate. It can drop or delay messages, which is an omission fault of the kind the rest of the system already handles.
- **One extra hop.** A message from a worker to its scheduler goes through the broker, not directly. Placing the broker in the provider region nearest the cluster keeps this short.
- **A new provider component to keep up.** The broker is a single point of failure unless it is clustered, which production brokers routinely are. Control traffic is light, about 0.46 KB/s per worker in a scalability pilot, or roughly 14 KB/s for a cluster of 30.

Table 2. Three ways to place the scheduler. Only the first is evaluated in this paper.

	Volunteer, per cluster	Provider, per cluster or region	Provider, global
How workers reach it	through the broker, by cluster topic	dial out to its address	dial out to its address
New provider component	broker	scheduler service	scheduler service
Scheduler failure	epoch fencing and succession	provider high availability	provider high availability
Scheduler-host adversary	in the threat model	none	none
Extra hop per request	yes	no	no
Scheduling domain	tens of hosts	chosen freely	whole fleet

6.4 The alternative: a provider-hosted scheduler

If the scheduler is global or regional rather than per cluster, none of this is needed. It runs on provider infrastructure at a stable address, workers dial out to it directly, there is no broker, and failover becomes ordinary high availability. The same planner and placer code runs unchanged. We still study the volunteer-hosted design, for three reasons. The provider then runs no per-cluster component, so its tier grows with traffic and fleet size, not with the number of clusters. It is the harder case for fault tolerance, so the mechanisms of Section 7 are tested where they matter. And the pattern of attested code, soft state and a provider referee may carry over to other coordinators on unreliable nodes.

Does the volunteer's compute matter? Using the planning times we measured, hosting every cluster's scheduler on the provider would cost about 0.9 cores for greedy planning at 10^5 workers (clusters of 30), and about one core for placement at 1 000 invocations per second. The exact planner would need about 450 cores. Volunteer hosting saves compute only for the exact planner, whose benefit we could not establish (Section 9.3), so the choice of greedy planning does not depend on where the scheduler runs. Provider hosting would also remove a round trip from every synchronous invocation, remove scheduler-host adversaries from the threat model, take the scheduler code out of the attested image, and allow scheduling domains larger than a volunteer machine can serve, in line with our geo-clustering result (Section 9.6). Table 2 compares the options.

6.5 What the simulator models

The simulator models only network delay. A message travels for a time derived from the distance between cities, plus a fixed extra 20 ms for each endpoint that is behind a relay, which we set for 15% of workers by default. That extra delay stands in for the broker hop. It does not model broker capacity, connection set-up, address changes or reconnection after a scheduler change. A design that sent every control message through a broker would add a hop to all workers, not 15%. We have not swept this. We therefore claim the architecture as a design, with delay modelled, and not a measured performance of any protocol. A direct comparison of the three placements, with a load that stresses a single scheduler and a sweep of the relayed share, is future work (Section 10.2).

7 FAULT TOLERANCE AND ACCOUNTABILITY

By Property 1, one set of mechanisms must cover crashes, churn and malice. We now say how a failure is noticed, how a failed scheduler is replaced, and how work is accounted for.

7.1 Progress, not liveness

The hard failures are the ones where *liveness signals look healthy*. A *black-holing* host (a node that quietly swallows what it receives) keeps sending heartbeats while dropping the work it accepted. An on–off host builds a clean record and then defects. A drain notice may be a lie. So we adopt a rule: *liveness signals from untrusted machines are hints, and progress observed by trusted or attested parties is evidence.* The trusted observers are the routers. The attested observers are the worker agents and the scheduler code, although their *hosts* are still untrusted for availability.

Each detector watches an event that must happen if a component works, and blames a suspect when it does not. The router's *dispatch watchdog* suspects a scheduler when an enqueued invocation is not acknowledged within 1 s. The scheduler's *result watchdog* suspects a worker that accepted work but has not returned a result by the predicted 90th-percentile completion time, times 1.2, and at least 3 s. Repeated dispatch timeouts become a *cluster-stall* report, and a *decision-latency* limit on the scheduler's attested timestamps catches a scheduler that is alive but slow. A running record of each worker's omission rate, with a change-point test, catches selective and on–off behaviour. Heartbeats stay as a fast path for plain crashes and are never sufficient alone. A suspected worker gets no new work, its in-flight work is duplicated or retried, and it loses provisioned eligibility. A suspected scheduler triggers failover.

7.2 Replacing a scheduler without consensus

Once a failure is suspected, the scheduler must be replaced. The textbook way is a replicated state machine among the candidates, but that is slow, and it brings an agreement problem onto machines that may be adversarial. Voss avoids the problem for two reasons: the provider's orchestrator is a reliable referee, and a scheduler's state is *soft*. Figure 6 shows the sequence.

Appointment and fencing. The orchestrator appoints a scheduler by an atomic compare-and-swap on the cluster's *epoch* number in its database, like a lease [13]. The appointment binds the cluster, the epoch and the session-certificate identifier of the appointee, so a cloned appliance with a different session key is not appointed. Workers accept dispatches only from the current epoch and certificate and answer older epochs with a `STALE` reply, which fences the old scheduler off. A scheduler that cannot reach the orchestrator for one lease period stops dispatching on its own. A partitioned scheduler can therefore cause only duplicate work, which at-least-once execution allows. That is safe for idempotent functions. For a non-idempotent function, a duplicate sent within the lease period could repeat its effect, and the protocol does not prevent this (Section 10.1).

Soft state. A scheduler keeps nothing durable on its host. Undispatched invocations are held by routers, running work and confirmed provisioning by workers, and estimator state in a checkpoint at the control plane. After appointment the successor installs the new epoch and performs a *roll call*: it asks the workers for their running work, confirmed pins and keys and recent outcomes, rebuilds its view and re-plans from the checkpoint. Because nothing authoritative lives on the old host, a restarted appliance is a new instance that must be appointed again.

Succession. The orchestrator keeps a ranked list of candidates by survival, speed, centrality and assurance, with diversity across failure domains. Every attested worker can serve, because the scheduler code is in the image, so promotion needs no boot. Planned handovers flush a checkpoint first, and a *fake* drain by a malicious scheduler host just causes a clean handover. With the broker of Section 6.3, the successor takes over by subscribing to the cluster's scheduler topic.

Worker failures. When a session ends, keys and warm sandboxes vanish and the invocation is retried or duplicated elsewhere, preferring provisioned machines. A per-cluster retry budget and router spillover contain retry storms. Owner notices such as drain make the scheduler more careful but never excuse an omission in the estimators.

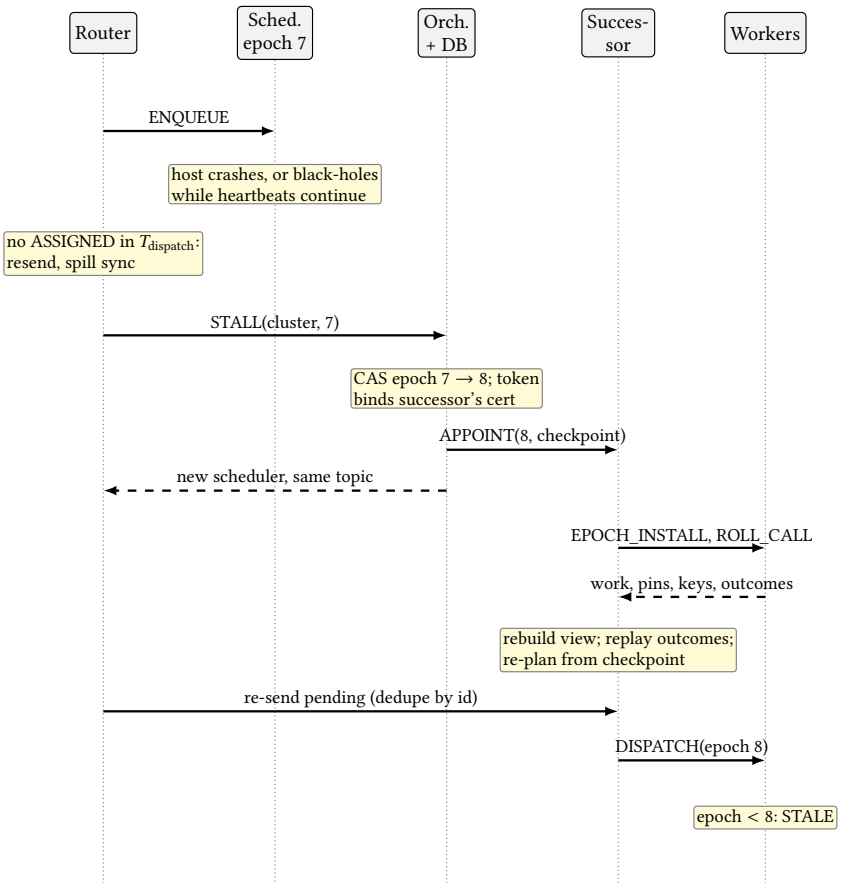


Fig. 6. Scheduler failover. The router's dispatch watchdog detects a crashed or black-holing scheduler. The orchestrator changes the epoch atomically and appoints a successor, which rebuilds soft state from workers and a checkpoint.

7.3 Receipts and credits

Each result carries a receipt signed with the session key. It names the invocation and attempt, the worker and certificate, the epoch, the execution time, the work done in reference units and a digest of the output. The router checks it against the certificate and the verifier's revocation list and appends it to the operational database. A machine cannot claim work it did not do. It can only decline to do work, which is an omission handled above. Credits are proportional to reference work, not elapsed time, so throttling earns nothing, and a small credit rewards cache that the plan asked a host to hold. Reputation is kept apart from the survival and omission estimators, so no signal is counted twice, and it is used only for rewards and as a tie-breaker. We leave out a blockchain on purpose. Receipts are already verifiable by third parties, and the only risk that remains is the control plane withholding records, which a signed transparency log would address better.

8 HOW WE EVALUATE

A volunteer platform with thousands of machines, controlled churn, hostile owners and paired comparisons of policies cannot be run on real hardware. We evaluate Voss in a deterministic

simulator and calibrate it against measurements. This section says what the simulator models, where its numbers come from, how far we checked it, and how every comparison is made.

8.1 The simulator

The simulator is a single-threaded discrete-event simulator in Python. Simulated time jumps from one event to the next in a fixed order, and every random draw comes from a named, seeded stream, so a run can be repeated exactly. A run takes a configuration, a calibration bundle and a seed. It first draws everything the system under test cannot influence: the machines, every session start and end, which owners are adversaries, the arrivals and the noise in each execution. These traces are content-hashed, so two systems compared on one seed face the same world. The engine then runs the actors (workers, schedulers, routers, orchestrator, verifier, key services) and the policies. Policies see only immutable views built from messages, never the world itself. At the end, checks assert that every success ran on an attested host after a key release, that each invocation has one outcome, and that the time components add up to the measured latency. Analysis reads only the raw tables.

Nothing is free. Every interaction is a message that arrives after a delay from the distance between about 100 cities (100 km per millisecond, times 1.3 for route detours), plus last-mile delay and jitter. Disk reads, CPU contention, TPM quotes, key release and the decision time of planner, placer, router and orchestrator all consume simulated time. Decision time is a fixed part plus a part proportional to the work, so results do not depend on the speed of the machine running the simulation. The coefficients are placeholders from the literature, not measurements. For the planner the charge is about 0.1 s, for the greedy and the exact planner alike, while the exact planner really takes 4.1 s, so its results are optimistic about how stale its plans are.

The world. Workers come in three tiers (nightly, idle-hours, dedicated) with different hardware, cities and trust substrates. Session lengths are heavy-tailed with time-of-day effects, most departures are abrupt, and correlated shocks remove a share of a city at once. No public trace of consumer donation sessions exists, so we follow volunteer-availability studies [31], mark the parameters *assumed* and sweep them. The workload has six tenants and 12 functions modelled on SeBS [17], in three service classes, with skewed popularity and bursty, day-and-night arrivals [43]. 11 of the 12 functions are marked idempotent, by our assumption. A fraction α of workers are adversaries who omit, delay, black-hole, defect after building a record, send fake drain notices, or collude within a failure domain. By Property 1 none of them forges results or reads keys on an attested substrate, while the unattested reference allows both.

8.2 Calibration and what we checked

Parameters from the specification are literature values or assumptions and are tagged so. Measured values come only from calibration bundles. We measured one on a consumer laptop (Intel Core i3-7020U, a discrete TPM 2.0, a hard disk), which stands in for a donor machine. A harness reboots it into the appliance and records timestamps at about one-second resolution, and microbenchmarks cover TPM quotes, hashing, microVM boot and snapshots. Compared with our placeholders, reboot to kernel is slower (21 s against 15 s), the phase from kernel start to a running agent is far slower (31 s), and creating a 512 MB snapshot on a hard disk takes 6.7 s. From power-on, an attested worker takes 65 s (median of 9 boots, 44 s of it after kernel start). A warm-cache microVM boot takes 154 ms, close to published figures [2].

Two limits apply. The laptop is not the intended donor machine and lacks AMD SEV-SNP and Intel TDX, so confidential-VM parameters are literature values, which we sweep [27, 33, 35]. And the appliance ran in *host mode* (the measurement chain and quote on the host, microVMs through

Firecracker) rather than as a locked image on bare metal. The attested-session figure is a measured sum, not a measured attestation of the final image.

A first check against real execution (E01). To test the start-up and execution parts of the simulator, we ran the same scenarios in a single-box emulation on a lab node (Intel Core i7-12700, 125 GB, NVMe, Firecracker 1.10.1) and in the simulator, with a calibration measured on that node. The emulation uses real microVMs for boot, snapshot restore, dispatch and function execution, and keeps network, key release, attestation and queueing as models (16 donor slices, five functions, ten minutes per run, three seeds). Success rate agrees within 0.6 percentage points in every scenario, the mix of start-up levels is identical, and median synchronous latency agrees within 6–23%. The *shape* of the latency distribution does not meet our pre-set cut-offs, so every scenario is graded "partly" close and none "close". At light load the simulator's P99 is 3.7 to 4.1 times the emulation's, driven by a handful of cold invocations: the P99 of about 530 requests is the five slowest. E01 does not test the network, attestation, key release, session boot or failure detection at scale, and no multi-machine deployment has validated the simulator end to end. We therefore read differences between designs, not absolute latencies.

8.3 Protocol

Each experiment is a frozen specification, and every cell is run with ten seeds (three for the two largest scalability points). Unless stated, Voss uses the greedy planner and the default placer, router and hedger. Systems in a comparison share the world and the per-invocation random draws, so each comparison is *paired* by seed. For a metric we take the per-seed difference and report its mean with a 95% bootstrap interval [23]. Percentiles are computed per run, then averaged, never pooled.

Each hypothesis is written down before the run with a mechanical rule. It is *supported* if the interval excludes zero in the predicted direction by at least a stated margin, *weak* if it excludes zero but the effect is below the margin, *inconclusive* if the interval includes zero, and *refuted* if it excludes zero the other way. The manuscript's numbers and tables are generated from the result tables by a script that also fails if a result stops supporting the sentence it backs.

We report success, the share of deadlines met, median and P99 synchronous latency, *waste* (extra attempts per success), donor memory held warm, and key exposure. A system is *feasible* in a scenario cell if the lower 95% bound of its success is at least 0.99 and either its deadline-met share or its P99 meets a target. Feasibility maps then show *where* each design works, rather than an average.

Baselines run on the same simulator, candidates and cost model and differ only in the registered policy: no planning (provision on arrival), keep-alive with a time limit, a heartbeat-only detector, and $f = 0$ (no robustness). The *simple baseline* combines static regional clusters, nearest-region routing, least-loaded placement, keep-alive and no hedging. Commercial-style references (*Lambda-like*, *Azure-like*) are always-on datacenter hosts with per-function keep-alive and nearest-region routing, priced from list prices, on the same engine.

8.4 Reproducibility

Every number, table and figure in Section 9 is regenerated from raw run logs by committed scripts, and runs are bit-reproducible: re-executing a run gives raw tables with the same result hash, recorded in a manifest with the scenario and calibration hashes. Floating-point arithmetic repeats bit for bit only under stated conditions, because a last-bit difference can flip a near-tie in placement and send the whole trajectory elsewhere. We found that NumPy's maths functions use different code paths on CPUs with and without AVX-512, and a first verification re-run failed for exactly this reason. We now pin NumPy to the SSE4 baseline that every x86-64 CPU supports and record a canary hash of the maths functions in each manifest. The full sweep, 21 experiments and 2,366 runs, was then re-executed on independently provisioned machines, and 2,366 of 2,366 hashes matched.

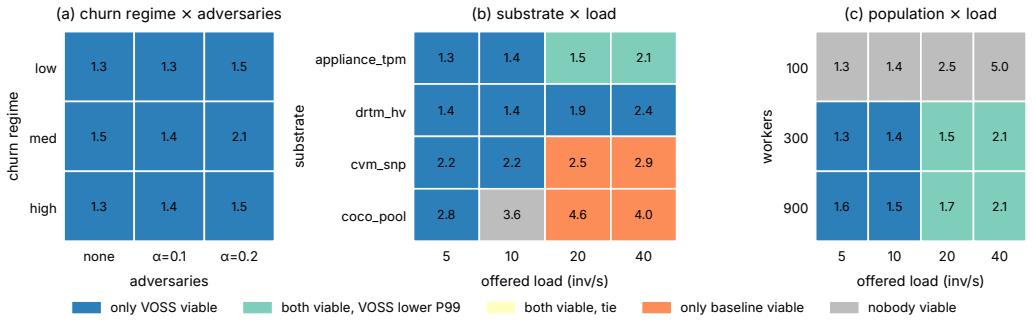


Fig. 7. Feasibility maps. Cell colour shows which of Voss and the best simple baseline meets the target (success at least 0.99 and 95% of deadlines met). Numbers are Voss's synchronous P99 in seconds. (a) churn against the fraction of omitting owners, (b) trust substrate against load in invocations per second, (c) population against load.

Reproduction needs an x86-64 CPU with fused multiply-add, Linux with glibc, Python 3.11 and the committed lockfile, and about four hours of wall-clock time as parallel jobs. No TPM or special hardware is needed. Appendix A gives the requirements, checks and commands.

9 RESULTS

We organise the results around the questions the design raises. Appendix B lists every hypothesis with its verdict.

9.1 What does trust cost, and how does it shape start-up?

The appliance pays a large one-off cost for each session, 19 s with our pre-measurement parameters and 65 s from power-on on the calibration machine. A confidential VM needs 1.4 s and a confidential-container pool 8.1 s. We had expected confidential VMs to be 10 to 100 times *slower* at establishing trust. That was refuted: they are about 93% faster, because the appliance's cost is a reboot. Once the session exists, though, the appliance is as fast as an unattested Firecracker host: warm and snapshot starts are within a few percent of the unattested figures (H5, supported), because attestation is paid per session, not per invocation.

9.2 Where does volunteer serverless work at all?

Averages hide the regimes in which a design fails, so we map feasibility. Figure 7 colours each cell by which of Voss and the best simple baseline is feasible (success at least 0.99 and 95% of deadlines met), and prints Voss's P99.

Voss is the only feasible system in all 9 cells of panel A, which spans three churn levels and up to 20% omitting owners, in 9 of 16 cells of panel B and in 4 of 12 cells of panel C. With an absolute P99 target of 3 s it is feasible in 9 of 9, 13 of 16 and 10 of 12 cells, and the simple baseline in none. At 5 s, Voss is feasible in 9, 16 and 10 cells.

The map also shows where Voss is the wrong choice. On confidential-VM substrates at higher load, the simple baseline is the only feasible system in 4 cells. We conjecture that with cheap session set-up there is little cold-start cost left for planning to remove, but we did not test this. With 100 workers no system is feasible in any cell of panel C, so the design needs a minimum population, and at a deadline-met target of 0.98 no system is feasible anywhere.

Table 3. Planner comparison (E04): mean ± 95% half-width over 10 seeds. Met: share of deadlines met; Warm: GB of idle warm memory.

Regime	Planner	P99 (s)	Met	Warm GB	Exposure	Solve (s)
churn_med	no planning	2.78±1.35	0.940±0.017	1±0	8.5±1.4	0.002±0.000
churn_med	keep-alive TTL	1.78±0.48	0.971±0.018	61±9	7.4±1.1	0.002±0.001
churn_med	greedy planner	1.29±0.09	0.978±0.015	94±14	19.9±1.8	0.008±0.001
churn_med	robust MILP	1.27±0.10	0.979±0.015	129±18	20.1±1.5	4.005±0.505
churn_high	no planning	3.48±1.32	0.923±0.029	1±0	7.5±1.2	0.001±0.000
churn_high	keep-alive TTL	3.28±1.91	0.940±0.030	59±10	7.3±0.9	0.001±0.000
churn_high	greedy planner	1.42±0.17	0.958±0.028	87±13	15.8±1.2	0.008±0.002
churn_high	robust MILP	1.31±0.09	0.958±0.031	119±11	15.2±0.9	4.164±0.650

9.3 Does planning attested session state pay?

This is the central claim, so we compare planners on identical scenarios (Table 3). The greedy planner lowers synchronous P99 by 54% at medium churn and 59% at high churn against no planning, and by 27% and 57% against a keep-alive timer. The exact planner gives 54% and 63% against no planning, and 29% and 60% against keep-alive (H1, supported). Planning costs donor memory: the exact planner holds about 2.1 times the warm memory of keep-alive at medium churn.

The extra gain from the exact solver is small and not established. At medium churn it improves P99 by 1.8% over the greedy planner, below our 5% threshold. At high churn its point estimate is 8.1%, above the threshold, but the 95% interval runs from a 21% reduction to a 1% increase and includes zero (H8, inconclusive). Ten seeds can neither confirm that the exact solver matters nor show that it does not. Its solve takes 4.1 s per cluster against 8 ms, yet the simulated clock charges both about 0.1 s. Our rule required the gain to be below 5% in every regime, and the data do not show that at high churn, so we do not claim the rule is met. We keep greedy as the default, a choice made before these runs.

To see which components matter, Figure 8 removes one at a time (10% omitting owners). Removing lifecycle-aware placement costs 3.3 s of P99, removing planning 1.1 s, removing robust coverage 0.8 s and removing hedging 0.3 s. The components that buy P99, planning and robustness, also raise exposure: without planning, total exposure is lower by 31 units (Eq. 1), because fewer hosts hold keys. Geo-clustering is the only component whose removal *improves* P99 (by 0.09 s) and also cuts exposure (by 34 units). We return to this in Section 9.6.

9.4 Does progress-based detection make malice look like churn?

Scheduler faults. Table 4 injects crashes, partitions, black holes and slowdowns into scheduler hosts. Heartbeats alone cannot see a black hole, so detection waits for a 60 s timeout (60 s) and the run suffers 132 timeouts, against 4.2 s and 0.7 with progress-based detection (H-FT1, supported). For crashes, detection takes 2.2 s and failover 2.7 s. A slowed but live scheduler is the hardest case: failover takes 14 s because evidence must build up before a slowdown can be told from load.

Omitting owners. Table 5 raises the fraction α of omitting owners. At $\alpha = 0.2$, Voss keeps 99.95% of requests succeeding, while heartbeat-only detection with otherwise identical machinery falls to 96.8% (H4, supported) and loses 9 times more results to omitting owners. Success under omission stays near the churn-only level, which supports the claim that attested omitting owners behave like churn once detection uses progress. The robust-coverage constraint is not what keeps success. Setting $f = 0$ changes success by only 0.11 percentage points (H2, inconclusive), though it lengthens P99. Detection and re-dispatch carry success; robust planning buys tail latency.

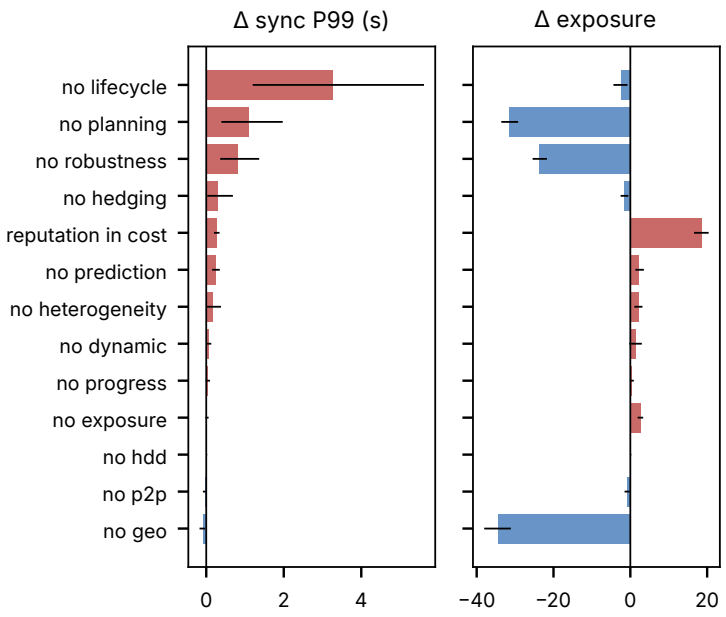


Fig. 8. Ablations (E19): change when one component is removed, relative to full Voss (paired, 95% intervals). Left: synchronous P99. Right: key exposure.

Table 4. Scheduler faults (E15): time to detection and failover, success, and P99 inside the fault window.

Fault	Detector	Detect (s)	Failover (s)	Success (%)	P99 in window (s)
crash	progress	2.2±0.3	2.7±0.4	99.95±0.02	1.28±0.15
crash	heartbeat_only	5.0±0.0	5.5±0.5	99.87±0.04	1.66±0.33
partition	progress	2.2±0.3	2.6±0.4	99.95±0.02	1.28±0.16
partition	heartbeat_only	5.0±0.0	5.5±0.5	99.87±0.04	1.67±0.31
blackhole	progress	4.2±3.0	4.6±3.0	99.95±0.02	1.28±0.15
blackhole	heartbeat_only	60.0±0.0	59.3±1.1	96.81±0.51	2.90±0.08
slow	progress	13.9±7.9	14.3±8.2	99.95±0.03	1.34±0.14
slow	heartbeat_only	60.0±0.0	59.3±1.1	99.55±0.41	2.00±0.33

Hedging. Always duplicating cuts P99 but wastes 10.6% extra attempts. Risk-proportional hedging wastes 1.8%, and its P99 is 13% lower at $\alpha = 0.1$, with an interval from -32% to +1% (H-FT3, inconclusive). It is cheap insurance, not shown to be necessary.

9.5 What do robustness and exposure cost?

Robustness f and the exposure weight trade latency against key exposure (E23). Exposure grows with f (13, 19 and 24 at $f = 0$, 0.1 and 0.2), as expected, since covering any f failures needs more key-holding hosts. Aggregate P99 stays within 1.27–1.35 s across configurations, so the knee at $f = 0.1$ saves 25% of exposure against the best-latency point for 1.6% more P99 (H3, supported). Per tenant class the picture is less benign. Low-sensitivity tenants stay at about 1.4 s or below, but for

Table 5. Omitting owners (E08): mean ± 95% half-width over 10 seeds. Omissions are results dropped by adversaries.

Adversaries	System	Success (%)	P99 (s)	Waste (%)	Omissions
none α=0	VOSS	99.99±0.01	1.27±0.11	1.7±0.5	0±0
none α=0	VOSS, f=0	99.99±0.01	1.60±0.24	2.2±0.6	0±0
none α=0	heartbeat-only	99.98±0.01	1.36±0.15	2.0±0.7	0±0
mixed α=0.1	VOSS	99.95±0.04	1.32±0.12	2.2±0.5	38±14
mixed α=0.1	VOSS, f=0	99.96±0.03	2.15±0.60	2.9±0.6	37±11
mixed α=0.1	heartbeat-only	98.19±0.75	1.36±0.13	4.0±0.9	298±115
mixed α=0.2	VOSS	99.95±0.03	1.43±0.16	2.6±0.5	61±13
mixed α=0.2	VOSS, f=0	99.83±0.17	2.05±0.52	3.5±0.8	80±31
mixed α=0.2	heartbeat-only	96.79±1.13	1.39±0.16	5.6±1.3	527±161
all α=0.2	VOSS	99.94±0.05	1.37±0.10	2.3±0.7	31±9
all α=0.2	VOSS, f=0	99.96±0.03	2.28±1.00	2.7±0.5	28±8
all α=0.2	heartbeat-only	97.70±1.26	1.42±0.14	5.2±2.4	385±227

high-sensitivity tenants P99 reaches 1.8 s in one robust configuration. The aggregate knee should not be applied to every tenant class, and each needs its own operating point.

9.6 Which components do not earn their complexity?

Geo-clustering. We compare one global scheduler with static regions, static k-means and dynamic clusters on 600 workers (about 225 of them attested) at 20 invocations per second, with clusters of about 20 to 35 attested hosts. The global scheduler plans over at most 40 hosts, which is part of why it holds fewer keys, and nothing in this load stresses one scheduler's capacity. Dynamic clusters (E05) have *higher* P99 than the global scheduler by 17% in Europe, 17% in Europe and North America and 5% worldwide (H7, refuted in all three), and hold 5.1 times the exposure in Europe, since each of the 9 clusters must cover demand with its own key holders. We attribute this to fewer hosts per cluster for pooling warm state and keys, but did not isolate the mechanism. We found no population in which geo-clustering wins at this scale.

Routing, caching, reputation. Without flash crowds, nearest-region routing meets 0.991 of deadlines and least-loaded routing 0.911, because it ignores distance. Cost-aware caching with peer fetch and a disk tier is at most 3% better than plain LRU in P99, and the intervals include zero (H6, inconclusive); all variants start about 98% of invocations from a warm sandbox, which leaves little for disk tiers to improve. Receipts stay off the critical path (H10, supported): lengthening settlement from 10 to 300 s changes P99 by at most ±1.5%, and the stream is small, about 17 rows and 4.5 KB per second at 20 invocations per second. Reputation adds little (2.4% lower P99, weak), and credit is highly concentrated (Gini 0.93; the top 10% of workers earn 93% of credit and 65% earn almost none), which is a fairness problem for an incentive that relies on credits.

9.7 How does Voss compare with baselines and a commercial reference?

Simple baseline. On a homogeneous, stable, benign pool we expected the simple baseline to match Voss (E10). It does not: Voss has 64% lower P99 (H11, refuted), which we attribute to planning's pre-warming against churn. The baseline holds about half of Voss's key exposure, though (Voss is 1.9 times higher). Whether the latency is worth the exposure is a policy choice that the trade-off front of the previous section lets a deployer make.

Commercial reference. On the same workload at list prices (E17), Voss's P99 at 20 invocations per second is 14% and its median 45% above the Lambda-like reference. Cost per million invocations,

including the control plane and excluding volunteer credits, falls from 20 USD at 10 invocations per second to 10 at 20 and 5 at 40 as the fixed tier is amortised, against about 12 USD for the references. Voss is cost-competitive only above roughly 20 invocations per second on this workload and never has lower latency.

Scale. We grow the population from 10^2 to 10^5 workers at constant load per worker in a cheaper mode (coarse heartbeats, greedy planning, two simulated minutes, E18). Up to 10^4 workers success is 99.8%. At 3×10^4 it falls to 97% with a decision P99 of 14 s, and at 10^5 to 81% with a synchronous P99 of 28 s and a decision P99 of 30 s. Decision latency grows from about 0.1 s to tens of seconds, while control traffic per worker stays flat (0.70–1.08 messages per second), so the limit lies in the central decision path and not in message volume. We did not isolate which component saturates first, and the two largest points use three seeds.

9.8 Do the conclusions depend on our calibration?

We repeated the key comparison under the placeholder and laptop-measured bundles (E24). Voss's P99 is 1.17 s with measured parameters against 1.32 s with placeholders, while the simple baseline moves from 3.9 s to 4.6 s. The advantage of planning therefore grows under measured parameters. Absolute values should not be read as predictions.

9.9 Summary of verdicts

Of the hypotheses stated before the runs, 11 are supported, 2 weak, 6 inconclusive and 5 refuted. The refuted ones (geo-clustering, the simple baseline on homogeneous hosts, the cost ratio of confidential VMs) and the inconclusive ones (the robust constraint for success, the exact solver, caching, hedging) are as informative as the supported ones.

10 DISCUSSION

What the evidence supports. The central claim holds in simulation: managing attested session state as a planned, perishable resource beats reactive policies. Planning cuts tail latency by roughly half under churn, lifecycle-aware placement and planning are the two largest contributors in the ablation, and progress-based detection is what keeps success high when owners omit. These statements concern mechanisms and are backed by paired intervals. They hold across churn levels and adversary fractions in the feasibility maps, not at one operating point.

What it does not. Four expected benefits did not appear. Geo-clustering is slower than a single scheduler and holds several times the exposure at our scale. The exact solver is not demonstrably better than the greedy planner, so its cost is justified only by the coverage guarantee, which we could not show matters for success. Caching sophistication and reputation change P99 by a few percent at most, and credit is extremely concentrated. A commercial-style reference is faster, and Voss is cheaper per invocation only once its fixed control plane is amortised. None of this contradicts the premise that volunteer hardware can serve latency-sensitive functions. It narrows which components earn their complexity. A leaner Voss (attestation, a greedy robust planner, a cost-based placer with hedging, progress-based detection, epoch-fenced failover and receipts) is the version our evidence favours.

Exposure is the price of latency. Planning, robustness and warm state all enlarge the set of machines holding a tenant's key. The simple baseline has about half of Voss's exposure, and moderate robustness captures most of the benefit in aggregate but not for every tenant class. A deployment should offer exposure and robustness as per-tenant policy, not global constants. The feasibility maps add that the trust substrate and a minimum population matter more than tuning: on confidential-VM substrates the simple baseline can suffice at high load, and below a few hundred hosts no system is feasible.

Reliance on attestation. Every robustness result uses omission and timing faults only. If attestation failed, for example through the physical attacks we excluded, adversaries could forge results and read keys, and exposure would stop being a proxy for risk.

Where the scheduler runs. Section 6 argued that a volunteer-hosted scheduler needs a broker and a provider-hosted one does not. The planner and placer results do not depend on the scheduler's location, except through one router-to-scheduler round trip and decision latency. The scheduler-fault results are specific to volunteer hosting, and the geo-clustering result suggests that larger scheduling domains, which a provider-hosted scheduler can offer, are better. We did not run the provider-hosted variant. We expect the planner and placer conclusions to carry over and the fault-tolerance results not to, but that is a prediction.

Function workflows. We treat invocations as independent, but many applications are graphs of functions in which one completion triggers the next [16], and each stage can pay a cold start in turn [20]. A trust-aware scheduler could prepare the keys and warm state of a function's successors while it runs. The risk is speculation: warming a function that a branch never calls wastes memory and may widen exposure, so look-ahead warming should be budgeted like hedging. We do not evaluate this.

10.1 Limits of this study

- **Simulation, partly validated.** No multi-machine deployment has validated the simulator. The single-box emulation (E01) agrees on success and start-up levels but not on the shape of the latency distribution, and it did not exercise the network, attestation, key release, session boot or failover. Policies are the real code, while the world (network, churn, adversaries) is a model.
- **Calibration scope.** One consumer laptop with a hard disk, with the appliance in host mode. Confidential-VM and hypervisor parameters are literature values. The main comparison survives a change of bundle (E24), but another CPU, disk or firmware could shift absolute costs.
- **Assumed workload and churn.** No public trace of donation sessions exists. The workload has 12 functions of six tenants, so the planner's hot-set limits never bind and the long tail of rarely used functions is not exercised. Functions make no calls to external services. 11 of 12 are marked idempotent by us, and the retry, hedging and success results under omission rest on that. This matches what providers already ask of developers (Section 2), but we did not measure how many real functions meet it. The adversary uses a fixed menu of strategies, and an adaptive one could do worse.
- **Idealised knowledge of costs.** The scheduler uses calibrated start-up, trust and network times and takes declared host speeds as exact, whereas the simulated world adds noise. We did not sweep prediction error.
- **Network and decision costs.** Network delay is geographic plus 20 ms for 15% of workers behind a relay. Broker capacity, connection set-up, address changes and reconnection are not modelled, and decision latency is a placeholder formula.
- **Scale and statistics.** The two largest scalability points use three seeds and a cheaper mode, and the geo-clustering comparison is too small to stress one scheduler. Ten seeds give wide intervals for tail metrics, and many experiments were run, so a single verdict should be read with that in mind. Literature parameters are quoted as reported and tagged for re-verification.

10.2 Future work

The most valuable next steps are to validate the simulator on a small real testbed, which would also test the host-mode assumption; to compare the three scheduler placements of Table 2 under a load

that stresses one scheduler, with a sweep of the relayed share; to measure confidential-VM hosts and refit decision costs; to revisit geo-clustering at larger populations; to evaluate workflow-aware warming; and to study adaptive adversaries and the concentration of credit.

11 RELATED WORK

Five lines of work each address part of what Voss needs: untrusted hosts, churn, latency targets, planned trust and warm state, gray-failure detection and verifiable accounting. None covers them together.

Volunteer computing. BOINC-style systems harvest idle machines for scientific batch work [6, 7]. Because hosts are untrusted, they replicate tasks and accept a result when a quorum agrees, and they use long deadlines to tolerate machines that disappear. Replication multiplies cost, presumes deterministic tasks without side effects, protects integrity but not the confidentiality of inputs, and works on a time scale of hours. Voss instead removes the integrity and confidentiality concern with attestation, so redundancy is needed only against omission, and then only for risky invocations.

Serverless scheduling and keep-alive. The cold-start literature decides when to keep sandboxes alive or pre-warm them [43], treats keep-alive as caching [24], or speeds up start-up with lighter sandboxes and snapshots [2, 8, 14, 36]. Low-latency schedulers place short tasks by sampling [38], and cluster managers pack tasks onto fleets [45]. All assume trusted, effectively always-on machines. A warm sandbox is lost only by a policy decision, never because an owner closed the lid, and there is no resource corresponding to a provisioned secret. Voss keeps the start-up vocabulary but adds a per-session trust step and plans warm state under churn and exposure constraints.

Harvested and edge serverless. Serverless platforms on harvested datacenter VMs handle evictions [47], and edge schedulers place containers near data and users [39]. Their hosts are run by the provider or a trusted party, so the failure model is eviction, not silent omission by an adversary, and no secret is entrusted to a party that might read it.

Confidential serverless. Trusted execution environments protect code from the host operating system and the platform operator [5, 9, 18], and recent work targets serverless functions in confidential VMs and containers [1, 35, 41]. These designs address confidentiality on provider-managed servers. They do not address machines whose sessions end unpredictably, and they do not treat establishing trust as a scheduled resource with a cost, a lifetime and an exposure. Voss is substrate-agnostic: the same scheduler runs over a TPM-attested appliance or over confidential VMs.

Trusted components and failure detection. Small trusted components, such as a counter or log, restrict what a faulty node can do and reduce the replication needed for Byzantine agreement [32, 46]. We use the idea differently: attestation does not make agreement cheaper, it makes the faulty node *non-Byzantine*, which lets us replace agreement among volunteers by a provider-run referee. Heartbeat and accrual detectors [26] infer liveness, which is not progress. Gray failures [28] motivate watching end-to-end progress, which Voss does at the trusted router and the attested scheduler.

Hedging and robust provisioning. Hedged requests reduce tail latency by duplicating slow work [21]. Voss hedges in proportion to estimated risk, on a different failure domain, within a budget. For the planner we use budgeted robust optimisation [11], which fits the question "is demand still covered if any f hosts fail together?" without a probability model for adversaries.

12 CONCLUSION

Latency-sensitive serverless computing on volunteer machines needs more than redundancy and keep-alive. Churn destroys the state that makes short functions fast, and untrusted owners make that state dangerous to create. We showed that attestation of a locked appliance reduces hostile owners to the same omissions and delays that churn causes, and that this exposes the real scheduling

problem: managing attested session state, which is slow to create, perishable and exposing. Voss answers with a slow planner and a fast placer, progress-based detection of gray failures, and epoch-fenced failover without consensus among volunteers, and it reaches volunteer-hosted schedulers through a provider-hosted publish/subscribe broker, with a provider-hosted scheduler as the simpler alternative. In a calibrated simulator it is feasible in regimes where simple baselines are not (9 of 9 and 13 of 16 cells at a 3 s target), it lowers tail latency by 54–59% against no planning, and it keeps success at 99.95% with 20% omitting owners. The advantage is not universal. On confidential-VM substrates at high load the simple baseline is the only feasible system, and geo-clustering, sophisticated caching and reputation do not pay for themselves in our experiments. The exact solver's advantage over greedy planning is not established. The next step is a real multi-machine testbed, which would validate the simulator and the networking design that this paper only models as delay.

REFERENCES

[1] 2025. Confidential Serverless Computing (Wallet). arXiv:2504.21518; USENIX NSDI 2026. TODO verify authors and final title.

[2] Alexandru Agache, Marc Brooker, Alexandra Iordache, Anthony Liguori, Rolf Neugebauer, Phil Piwonka, and Diana-Maria Popa. 2020. Firecracker: Lightweight Virtualization for Serverless Applications. In *Proc. USENIX NSDI*. 419–434.

[3] Amazon Web Services. 2026. How Lambda handles errors and retries with asynchronous invocation. AWS Lambda Developer Guide, https://docs.aws.amazon.com/lambda/latest/dg/invocation-async-error-handling.html. TODO verify; accessed October 2026.

[4] Amazon Web Services. 2026. Understanding retry behavior in AWS Lambda. AWS Lambda Developer Guide, https://docs.aws.amazon.com/lambda/latest/dg/invocation-retries.html. TODO verify; accessed October 2026.

[5] AMD. 2020. SEV-SNP: Strengthening VM Isolation with Integrity Protection and More. White paper.

[6] David P. Anderson. 2004. BOINC: A System for Public-Resource Computing and Storage. In *Proc. IEEE/ACM Int. Workshop on Grid Computing*. 4–10.

[7] David P. Anderson, Jeff Cobb, Eric Korpela, Matt Lebofsky, and Dan Werthimer. 2002. SETI@home: An Experiment in Public-Resource Computing. *Commun. ACM* 45, 11 (2002), 56–61.

[8] Lixiang Ao, George Porter, and Geoffrey M. Voelker. 2022. FaaSnap: FaaS Made Fast Using Snapshot-based VMs. In *Proc. EuroSys*. 730–746.

[9] Sergei Arnautov, Bohdan Trach, Franz Gregor, Thomas Knauth, Andre Martin, Christian Priebe, Joshua Lind, Divya Muthukumaran, Dan O'Keeffe, Mark L. Stillwell, David Goltzsche, Dave Eyers, Rüdiger Kapitza, Peter Pietzuch, and Christof Fetzer. 2016. SCONE: Secure Linux Containers with Intel SGX. In *Proc. USENIX OSDI*. 689–703. TODO verify.

[10] Algirdas Avižienis, Jean-Claude Laprie, Brian Randell, and Carl Landwehr. 2004. Basic Concepts and Taxonomy of Dependable and Secure Computing. *IEEE Trans. Dependable and Secure Computing* 1, 1 (2004), 11–33. TODO verify.

[11] Dimitris Bertsimas and Melvyn Sim. 2004. The Price of Robustness. *Operations Research* 52, 1 (2004), 35–53.

[12] Henk Birkholz, Dave Thaler, Michael Richardson, Ned Smith, and Wei Pan. 2023. *Remote ATtestation procedureS (RATS) Architecture*. RFC 9334. IETF. TODO verify.

[13] Mike Burrows. 2006. The Chubby Lock Service for Loosely-Coupled Distributed Systems. In *Proc. USENIX OSDI*. 335–350. TODO verify.

[14] James Cadden, Thomas Unger, Yara Awad, Han Dong, Orran Krieger, and Jonathan Appavoo. 2020. SEUSS: Skip Redundant Paths to Make Serverless Fast. In *Proc. EuroSys*. 32:1–32:15.

[15] Cloudflare. 2022. Introducing Cloudflare Queues. Cloudflare blog, https://blog.cloudflare.com/introducing-cloudflare-queues/. TODO verify.

[16] Marcin Copik, Marcin Chrapek, Larissa Schmid, Alexandru Calotoiu, and Torsten Hoefler. 2024. SeBS-Flow: Benchmarking Serverless Cloud Function Workflows. arXiv preprint. TODO verify authors, venue and year.

[17] Marcin Copik, Grzegorz Kwasniewski, Maciej Besta, Michal Podstawski, and Torsten Hoefler. 2021. SeBS: A Serverless Benchmark Suite for Function-as-a-Service Computing. In *Proc. ACM/IFIP Middleware*. 64–78.

[18] Victor Costan and Srinivas Devadas. 2016. Intel SGX Explained. IACR Cryptology ePrint Archive, 2016/086.

[19] Flaviu Cristian. 1991. Understanding Fault-Tolerant Distributed Systems. *Commun. ACM* 34, 2 (1991), 56–78. TODO verify.

[20] Nilanjan Daw, Umesh Bellur, and Purushottam Kulkarni. 2020. Xanadu: Mitigating Cascading Cold Starts in Serverless Function Chain Deployments. In *Proc. ACM/IFIP Middleware*. 356–370. TODO verify.

[21] Jeffrey Dean and Luiz André Barroso. 2013. The Tail at Scale. *Commun. ACM* 56, 2 (2013), 74–80.

[22] John R. Douceur. 2002. The Sybil Attack. In *Proc. Int. Workshop on Peer-to-Peer Systems (IPTPS)*. 251–260.

[23] Bradley Efron and Robert J. Tibshirani. 1993. *An Introduction to the Bootstrap*. Chapman & Hall.

[24] Alexander Fuerst and Prateek Sharma. 2021. FaaSCache: Keeping Serverless Computing Alive with Greedy-Dual Caching. In *Proc. ACM ASPLOS*. 386–400.

[25] Google Cloud. 2026. Configure event-driven function retries. Cloud Run functions documentation, https://docs.cloud.google.com/functions/docs/writing/retries. TODO verify; accessed October 2026.

[26] Naohiro Hayashibara, Xavier Défago, Rémy Yared, and Takuya Katayama. 2004. The φ Accrual Failure Detector. In *Proc. IEEE Symp. on Reliable Distributed Systems (SRDS)*. 66–78.

[27] Benjamin Holmes, Jason Waterman, and Dan Williams. 2024. SEVeriFast: Minimizing the Root of Trust for Fast Startup of SEV Microvms. In *Proc. ACM ASPLOS*. TODO verify title and pages.

[28] Peng Huang, Chuanxiong Guo, Lidong Zhou, Jacob R. Lorch, Yingnong Dang, Murali Chintalapati, and Randolph Yao. 2017. Gray Failure: The Achilles' Heel of Cloud-Scale Systems. In *Proc. Workshop on Hot Topics in Operating Systems (HotOS)*. 150–155. TODO verify.

[29] Qi Huangfu and J. A. Julian Hall. 2018. Parallelizing the Dual Revised Simplex Method. *Mathematical Programming Computation* 10, 1 (2018), 119–142.

[30] Hans Niklas Jacob, Christian Werling, Robert Buhren, and Jean-Pierre Seifert. 2023. faulTPM: Exposing AMD fTPMs' Deepest Secrets. In *Proc. IEEE European Symp. on Security and Privacy (EuroS&P)*. TODO verify.

[31] Bahman Javadi, Derrick Kondo, Alexandru Iosup, and Dick Epema. 2013. The Failure Trace Archive: Enabling the Comparison of Failure Measurements and Models of Distributed Systems. *J. Parallel and Distributed Computing* 73, 8 (2013), 1208–1223.

[32] Dave Levin, John R. Douceur, Jacob R. Lorch, and Thomas Moscibroda. 2009. TrInc: Small Trusted Hardware for Large Distributed Systems. In *Proc. USENIX NSDI*. 1–14.

[33] Dingji Li, Zeyu Mi, Chenhui Ji, Yifan Tan, Binyu Zang, Haibing Guan, and Haibo Chen. 2023. Bifrost: Analysis and Optimization of Network I/O Tax in Confidential Virtual Machines. In *Proc. USENIX ATC*. TODO verify.

[34] Microsoft. 2026. Resilient Event Hubs and Functions design. Azure Architecture Center, https://learn.microsoft.com/en-us/azure/architecture/serverless/event-hubs-functions/resilient-design. TODO verify; accessed October 2026.

[35] Masanori Misono, Dimitrios Stavrakakis, Nuno Santos, and Pramod Bhatotia. 2025. Confidential VMs Explained: An Empirical Analysis of AMD SEV-SNP and Intel TDX. In *Proc. ACM SIGMETRICS*. TODO verify.

[36] Edward Oakes, Leon Yang, Dennis Zhou, Kevin Houck, Tyler Harter, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. 2018. SOCK: Rapid Task Provisioning with Serverless-Optimized Containers. In *Proc. USENIX ATC*. 57–70.

[37] OASIS. 2019. *MQTT Version 5.0, OASIS Standard*. Technical Report. OASIS. TODO verify.

[38] Kay Ousterhout, Patrick Wendell, Matei Zaharia, and Ion Stoica. 2013. Sparrow: Distributed, Low Latency Scheduling. In *Proc. ACM SOSP*. 69–84.

[39] Thomas Rausch, Alexander Rashed, and Schahram Dustdar. 2021. Optimized Container Scheduling for Data-Intensive Serverless Edge Computing. *Future Generation Computer Systems* 114 (2021), 259–271. TODO verify.

[40] Nabil Schear, Patrick T. Cable, Thomas M. Moyer, Bryan Richard, and Robert Rudd. 2016. Bootstrapping and Maintaining Trust in the Cloud. In *Proc. Annual Computer Security Applications Conf. (ACSAC)*. 65–77.

[41] Carlos Segarra, Tobias Pfandzelter, et al. 2024. Serverless Confidential Containers: Challenges and Opportunities. In *Proc. Workshop on Serverless Systems, Applications and Methodology (SESAME)*. TODO verify authors.

[42] Glenn Shafer and Vladimir Vovk. 2008. A Tutorial on Conformal Prediction. *J. Machine Learning Research* 9 (2008), 371–421.

[43] Mohammad Shahrad, Rodrigo Fonseca, Iñigo Goiri, Gohar Chaudhry, Paul Batum, Jason Cooke, Eduardo Laureano, Colby Tresness, Mark Russinovich, and Ricardo Bianchini. 2020. Serverless in the Wild: Characterizing and Optimizing the Serverless Workload at a Large Cloud Provider. In *Proc. USENIX ATC*. 205–218.

[44] Trusted Computing Group. 2019. Trusted Platform Module Library Specification, Family 2.0. TCG specification. TODO verify revision.

[45] Abhishek Verma, Luis Pedrosa, Madhukar Korupolu, David Oppenheimer, Eric Tune, and John Wilkes. 2015. Large-scale Cluster Management at Google with Borg. In *Proc. EuroSys*. 18:1–18:17.

[46] Giuliana Santos Veronese, Miguel Correia, Alysson Neves Bessani, Lau Cheuk Lung, and Paulo Verissimo. 2013. Efficient Byzantine Fault-Tolerance. *IEEE Trans. Computers* 62, 1 (2013), 16–30.

[47] Yanqi Zhang, Íñigo Goiri, Gohar Irfan Chaudhry, Rodrigo Fonseca, Sameh Elnikety, Christina Delimitrou, and Ricardo Bianchini. 2021. Faster and Cheaper Serverless Computing on Harvested Resources. In *Proc. ACM SOSP*. 724–739.

A REPRODUCTION

Every result in this paper can be regenerated from raw logs. Table 6 lists what bit-for-bit reproduction needs. We checked it three ways. Before the NumPy pin, runners with and without AVX-512 disagreed on every run of one experiment (0 of 50 matched). After the pin, a probe experiment gave identical hashes on an Intel Xeon Platinum 8573C and on AMD EPYC 9V74, 9V45 and 7763 (the last without AVX-512), under Ubuntu 22.04 and 24.04. And the full sweep, re-executed on independently provisioned runners, matched 2,366 of 2,366 runs, with identical tables.

Table 6. Requirements to reproduce the results bit for bit.

CPU	x86-64 with fused multiply-add (every Intel and AMD CPU since 2013). AVX-512 is neither required nor used: the pin selects the SSE4 baseline, so AVX2 and AVX-512 machines agree.
Operating system	Linux with the GNU C library (tested: Ubuntu 22.04 and 24.04, glibc 2.35 and 2.39).
Software	Python 3.11 and the dependency versions in the committed lockfile (NumPy 2.x, SciPy with HiGHS, pandas, PyArrow); `NPY_DISABLE_CPU_FEATURES="X86_V4 X86_V3 AVX512_ICL AVX512_SPR"`, set by the package, the Makefile and the CI workflows.
Special hardware	None: no TPM, reboot or BIOS access. The calibration bundle is committed.
Compute	21 experiments, 2,366 runs. About four hours as 21 parallel jobs on hosted runners; compressed raw logs are 0.1–0.7 GB per experiment.
Not covered	ARM, other operating systems and other C libraries are not bit-comparable; for them the portable claim is statistical (ten paired seeds). Wall-clock columns vary by machine and are excluded from the hashes.

With the lockfile installed, the following regenerates the runs, compares them with the recorded hashes and rebuilds this document.

```
uv sync --frozen
make paper          # 21 specs x 10 seeds
make paper-verify   # compare hashes
voss repro runs/E09/<run>/manifest.json
make manuscript
```

B HYPOTHESIS SUMMARY

Table 7 lists every hypothesis, stated before the runs, with its verdict by the rules of Section 8.3. A dash marks cost-side information without a verdict.

Table 7. Hypotheses, paired relative change with 95% bootstrap interval, and verdict.

H	Claim	Exp.	Cell	Δ rel. [95% CI]	Verdict
H1	MILP planner cuts sync P99 vs no planning	E04	churn_med	-54.5% [-105.8, -12.6]	**supported**
H1	MILP planner cuts sync P99 vs no planning	E04	churn_high	-62.5% [-100.3, -27.0]	**supported**
H1	... and vs keep-alive TTL	E04	churn_med	-28.7% [-57.6, -7.7]	**supported**
H1	... and vs keep-alive TTL	E04	churn_high	-60.2% [-124.6, -12.2]	**supported**
H1	donor memory held warm: MILP vs keep-alive TTL (cost side)	E04	churn_med	+109.8% [+90.2, +125.8]	–
H2	f-robust plans keep success where non-robust collapse (α=0.2 mixed)	E08	mixed α=0.2	-0.1% [-0.3, +0.0]	inconclusive
H3	Pareto front has a knee; moderate exposure loses little latency	E23		+1.6%	**supported**
H4	progress detection beats heartbeat-only on success under α=0.2	E08	mixed α=0.2	-3.2% [-4.3, -2.1]	**supported**
H5	appliance warm start ≈ Firecracker without attestation (L3)	E-CS1 (sim)		-2.0%	**supported**
H5	appliance snapshot start ≈ Firecracker without attestation (L2)	E-CS1 (sim)		+2.5%	**supported**
H5	CVM classes are 10-100x slower than the appliance to establish trust	E-CS1 (sim)		-92.7%	**refuted**
H6	GDSF + P2P + HDD tier beat LRU without P2P/HDD (sync P99)	E09		+2.8% [-2.9, +11.9]	inconclusive
H6	... GDSF vs LRU alone	E09		+1.7% [-0.2, +4.1]	inconclusive
H7	geo clusters reduce sync P99 vs one global scheduler (EU)	E05	regions=[eu]	+17.3% [+12.5, +22.0]	**refuted**
H7	... EU+NA	E05	regions=[eu, na]	+16.6% [+5.2, +33.1]	**refuted**
H7	... world	E05	regions=[]	+4.5% [+0.9, +8.7]	**refuted**
H8	planner MILP beats greedy by ≥5% sync P99 (high churn)	E04	churn_high	-8.1% [-20.8, +1.1]	inconclusive
H9	distribution-aware (quantile) beats expectation-only on P99 at high churn	E03	churn_high, name=blr	-2.8% [-5.3, -0.8]	weak
H10	accounting is off the critical path: settlement period 10 s vs 300 s	E13	faults=[]	+1.5% [-0.2, +4.2]	**supported**
H10	... receipt batch 0.2 s vs 5 s	E13	faults=[]	-1.0% [-3.3, +1.0]	**supported**
H11	simple baseline matches VOSS when homogeneous, stable and benign	E10	hw=C1	+63.7% [+25.0, +112.8]	**refuted**
H-FT1	progress detection vs heartbeat-only: success with a black-holed scheduler	E15	faults=sched_blackhole	+3.2% [+2.7, +3.7]	**supported**
H-FT3	risk-proportional hedging (θ=.3, 5%) cuts sync P99 under α=0.1	E16	mixed α=0.1	-13.3% [-32.5, +1.4]	inconclusive
H-FT3	... at what extra-attempt cost	E16	mixed α=0.1	+253.0% [+182.6, +332.0]	–
RQ15	VOSS vs Lambda-like reference, sync P99 at 20 inv/s	E17	rate=20	+14.1% [+10.4, +18.3]	–
E12	reputation tie-break (beta-decay) improves sync P99 vs no reputation	E12		-2.4% [-4.4, -0.8]	weak
E12	... and success rate	E12		-0.0% [-0.0, +0.0]	inconclusive